Lab: RSA Using OpenSSL

Estimated Time Needed: 30 minutes

Overview:

In this lab, you will learn how to encrypt and decrypt files using RSA encryption with OpenSSL. This will involve generating RSA keys, encrypting a file using the public key, and decrypting it using the private key.

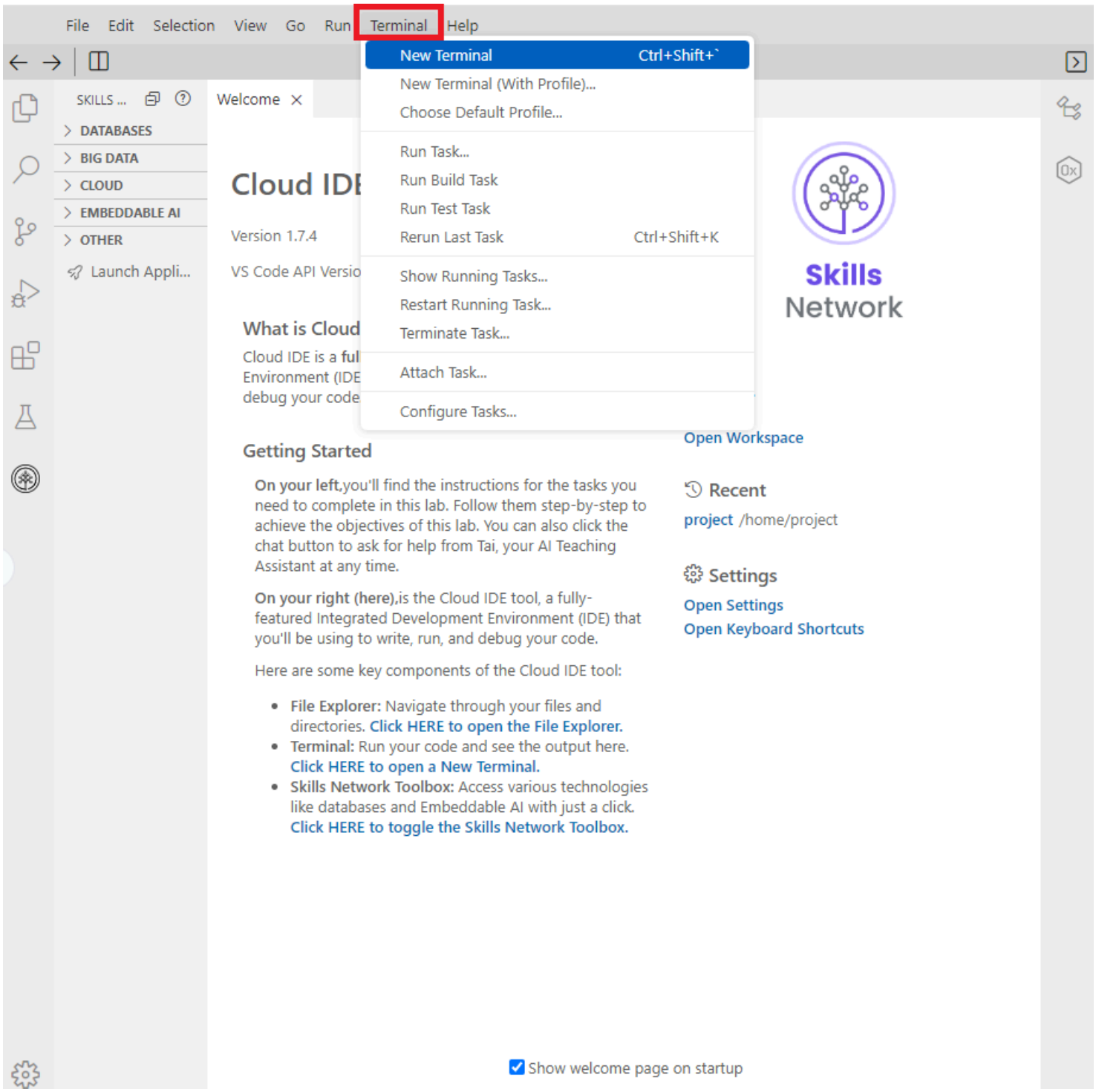
Learning objectives:
After completing this lab, you will be able to:

- Create RSA key pairs for encryption
- Encrypt and decrypt files using RSA

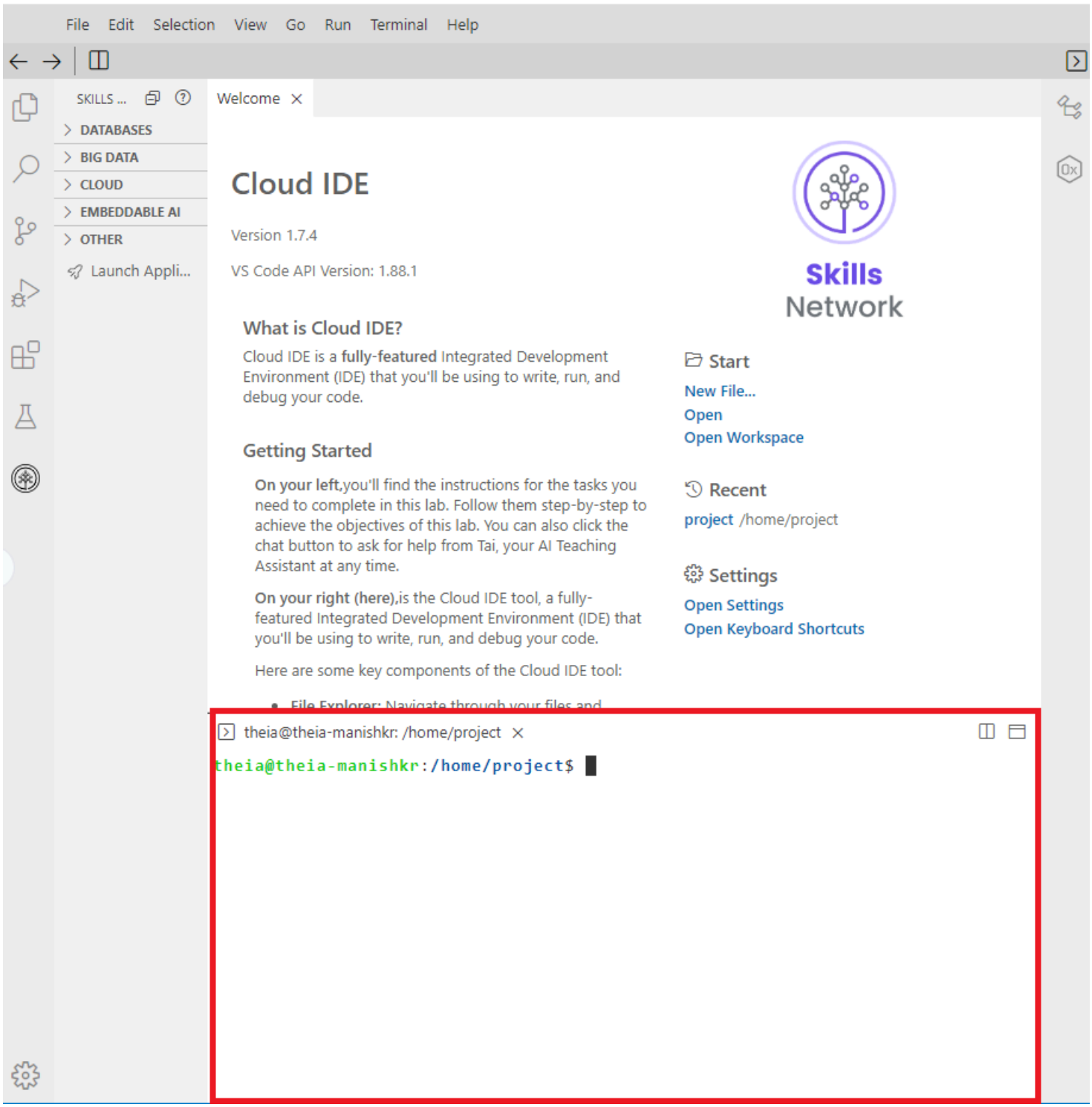
Prerequisites (optional):
Familiarity with using the Linux command prompt.

Initializing the lab environment

1. Click the Terminal menu and select New Terminal.



2. Copy and paste the commands into the terminal to complete the remaining steps.



Step 1: Generate RSA private key

```
openssl genpkey -algorithm RSA -out private_key.pem -pkeyopt rsa_keygen_bits:2048
```

Command breakdown:

- **`openssl genpkey`**: Generates a private key using OpenSSL.
- **`-algorithm RSA`**: Specifies RSA as the key generation algorithm.
- **`-out private_key.pem`**: Outputs the private key to a file named `private_key.pem`.
- **`-pkeyopt rsa_keygen_bits:2048`**: Sets the RSA key size to 2048 bits, a secure and commonly used key length.

Result:
This command creates a 2048-bit RSA private key and saves it in the PEM format.

Step 2: Extract public key from private key

```
openssl rsa -pubout -in private_key.pem -out public_key.pem
```

Command breakdown:

- **`openssl rsa`**: Invokes the RSA utility in OpenSSL to handle RSA key operations.
- **`-pubout`**: Extracts the public key from the given private key.
- **`-in private_key.pem`**: Specifies the input file containing the private key (`private_key.pem`).
- **`-out public_key.pem`**: Specifies the output file where the extracted public key will be saved (`public_key.pem`).

Result:
This command extracts the public key from the provided RSA private key and saves it in PEM format.

Step 3: Create a test file

```
echo "This is a test file for RSA encryption." > test_file.txt
```

Command breakdown:

- **`echo`**: A command used to display a string or text in the terminal.
- **`"This is a test file for RSA encryption."`**: The text string to be written into the file.
- **`>`**: Redirects the output of the `echo` command to a file, overwriting the file if it already exists.
- **`test_file.txt`**: The name of the file where the text will be written.

Result:
This command creates (or overwrites) a file named `test_file.txt` containing the text “This is a test file for RSA encryption.”

Step 4: Encrypt the test file using RSA public key

Step 4.1: Encrypt the file

```
openssl pkeyutl -encrypt -in test_file.txt -pubin -inkey public_key.pem -out test_file_encrypted.bin
```

Command breakdown:

```
openssl pkeyutl -encrypt -in test_file.txt -pubin -inkey public_key.pem -out test_file_encrypted.bin
```

- **`openssl`**: Invokes the OpenSSL tool for performing cryptographic operations.
- **`pkeyutl`**: A utility for public key operations, such as encryption, decryption, signing, and verification.
- **`-encrypt`**: Specifies that the input file should be encrypted.
- **`-in test_file.txt`**: Specifies the input file (`test_file.txt`) containing the plaintext data to be encrypted.
- **`-pubin`**: Indicates that the input key is a public key.
- **`-inkey public_key.pem`**: Specifies the public key file (`public_key.pem`) to be used for encryption.
- **`-out test_file_encrypted.bin`**: Defines the output file (`test_file_encrypted.bin`) where the encrypted data will be saved.

Result:

This command encrypts `test_file.txt` using the public key from `public_key.pem` and saves the encrypted output to `test_file_encrypted.bin`.

Step 4.2: Open and verify if that file is encrypted

```
cat test_file_encrypted.bin
```

Command breakdown:

- **`cat`**: A command to display the contents of a file.
- **`test_file_encrypted.bin`**: The file containing the encrypted data.

Result:

This command will display the raw, binary content of `test_file_encrypted.bin` in the terminal. Since the file contains encrypted binary data, the output may include unreadable characters and symbols, making it difficult to interpret visually.

Step 5: Decrypt the test file using RSA private key

Step 5.1: Decrypt the File

```
openssl pkeyutl -decrypt -in test_file_encrypted.bin -inkey private_key.pem -out test_file_decrypted.bin
```

Command breakdown:

- **`openssl`**: Invokes the OpenSSL tool.
- **`pkeyutl`**: A utility for public key cryptographic operations such as encryption, decryption, signing, and verification.
- **`-decrypt`**: Specifies that the operation is decryption.
- **`-in test_file_encrypted.bin`**: Indicates the input file containing the encrypted data.
- **`-inkey private_key.pem`**: Specifies the private key file used for decryption.
- **`-out test_file_decrypted.bin`**: Specifies the output file where the decrypted content will be saved.

Result:

This command decrypts the content of `test_file_encrypted.bin` using the private key in `private_key.pem` and outputs the decrypted content into `test_file_decrypted.bin`.

Step 5.2: Open and verify if that file is decrypted

cat test_file_decrypted.bin

Command breakdown:

```
cat test_file_decrypted.bin
```

- **`cat`**: A command-line utility used to display the contents of a file.
- **`test_file_decrypted.bin`**: The file whose contents will be displayed.

Result:

This command outputs the decrypted content of `test_file_decrypted.bin` to the terminal, allowing you to verify that the decryption process was successful and the original message has been restored.

Exercises

Exercise 1: Encrypt and decrypt a small message using RSA

Objective:
Learn to encrypt and decrypt a short message using RSA.

Task:

1. Generate an RSA key pair.
2. Create a text file with a short message.
3. Encrypt the file using the RSA public key.
4. Decrypt the file using the RSA private key and verify its content.

Hint:

- **Generate key pair:**
  Use the following commands:

  ```
  openssl genpkey -algorithm RSA -out private_key.pem -pkeyopt rsa_keygen_bits:2048
  openssl rsa -in private_key.pem -pubout -out public_key.pem
  ```

- **Create message:**
  Create a file `message.txt` with content such as :

  ```
  echo "The password is: secret123" > message.txt
  ```

- **Encrypt message:**
  Use `openssl pkeyutl` to encrypt:

  ```
  openssl pkeyutl -encrypt -in message.txt -pubin -inkey public_key.pem -out encrypted_message.bin
  ```

- **Decrypt message:**
  Use the private key to decrypt:

  ```
  openssl pkeyutl -decrypt -in encrypted_message.bin -inkey private_key.pem -out decrypted_message.txt
  ```

  Verify by viewing the decrypted content:

  ```
  cat decrypted_message.txt
  ```

Exercise 2: RSA key pair generation and file encryption

Objective:
Generate an RSA key pair and use it to encrypt and decrypt a file.

Task:

1. Generate an RSA key pair (2048 bits).
2. Create a plaintext file.
3. Encrypt the file using the RSA public key.
4. Decrypt it using the RSA private key and verify the output.

Hint:

- **Generate key pair:**
  Use:

  ```
  openssl genpkey -algorithm RSA -out private_key.pem -pkeyopt rsa_keygen_bits:2048
  openssl rsa -in private_key.pem -pubout -out public_key.pem
  ```

- **Create a new file:**
  Add content to `plaintext.txt`:

  ```
  echo "Confidential data for encryption." > plaintext.txt
  ```

- **Encrypt file:**
  Use:

  ```
  openssl pkeyutl -encrypt -in plaintext.txt -pubin -inkey public_key.pem -out encrypted_data.bin
  ```

- **Decrypt file:**
  Decrypt using:

  ```
  openssl pkeyutl -decrypt -in encrypted_data.bin -inkey private_key.pem -out decrypted_data.txt
  ```

  Confirm by comparing the files:

  ```
  cat decrypted_data.txt
  ```

Exercise 3: Simulate RSA key length impact on performance

Objective:
Analyze how varying RSA key lengths affect encryption and decryption time.

Task:

1. Generate RSA key pairs of different lengths (1024, 2048, 4096 bits).
2. Encrypt and decrypt a sample file using each key length.
3. Measure and compare the time taken for each operation.

Hint:

- **Generate keys:**

```
openssl genpkey -algorithm RSA -out private_key_1024.pem -pkeyopt rsa_keygen_bits:1024
openssl genpkey -algorithm RSA -out private_key_2048.pem -pkeyopt rsa_keygen_bits:2048
openssl genpkey -algorithm RSA -out private_key_4096.pem -pkeyopt rsa_keygen_bits:4096
```

- **Encrypt/Decrypt with timing:**
  Use `time` to measure:

```
time openssl pkeyutl -encrypt -in plaintext.txt -pubin -inkey public_key.pem -out encrypted_data.bin
```

```
time openssl pkeyutl -decrypt -in encrypted_data.bin -inkey private_key.pem -out decrypted_data.txt
```

Conclusion

In this lab, you successfully learned how to encrypt and decrypt files using RSA encryption with OpenSSL. By generating an RSA key pair, you were able to securely encrypt a file with the public key and decrypt it using the private key. This process demonstrated the fundamental principles of asymmetric encryption, where the encryption and decryption keys are different, ensuring secure data transmission. You can now apply these techniques to protect sensitive files and practice encryption in various real-world scenarios.

Author(s)

Dr. Manish Kumar